

Projet IA & Jeux 2025 – Paint Wars

Belkiss TISS (21231069) – Mohamed GUERZOU (21320883)

Objectif

Concevoir une équipe de 4 robots autonomes capables de maximiser la surface peinte dans une

arène compétitive, en respectant les contraintes du simulateur Tetracomposibot : aucune communication, une seule mémoire entière, aucune carte, toute la logique dans la fonction step.

Architecture

Nous avons implémenté une architecture de subsomption combinant des comportements réactifs

de type Braitenberg et un contrôleur neuronal simple. Les décisions sont hiérarchisées par priorité :

anti-blocage, anti-collision alliée, évitement des murs, évitement des robots, puis stratégie spécifique au robot.

Mémoire interne

La mémoire unique autorisée (self.memory) est utilisée comme un mini-automate : $\text{MODE} \times 10000$

+ TIMER. Elle permet d'activer temporairement des comportements comme un mode panique

lorsque le robot est bloqué.

Comportements utilisés

- hateWall : éviter les murs

- hateBot : évitement global des robots

- loveBot_ennemi : attraction vers les ennemis pour repeindre leur territoire

- Anti-collision alliée : règle déterministe basée sur l'identifiant du robot

Contrôleur neuronal

Un robot utilise un perceptron à deux sorties (translation, rotation) dont les poids ont été optimisés

préalablement par algorithme génétique. Il assure une navigation continue et efficace.

Spécialisation des robots

Robot 0 : navigation optimisée (Perceptron AG)

Robot 1 : chasseur d'ennemis

Robot 2 : explorateur tactique

Robot 3 : hybride murs/robots

Conclusion

Cette approche permet une bonne couverture de l'arène, limite les blocages et s'adapte aux arènes

inédites, tout en respectant strictement les règles du projet Paint Wars.

Fonctions utilisées et rôle

- `borner(valeur)` : borne une valeur entre -1 et 1 pour sécuriser les commandes moteur.
- `obtenir_id_autre_robot(i)` : extrait l'identifiant numérique d'un robot détecté par un capteur.
- `est_ennemi(i)` : teste si le capteur i détecte un robot adverse.
- `est_allie(i)` : teste si le capteur i détecte un robot de la même équipe.
- `separer_murs_robots()` : sépare les distances des murs et des robots dans deux tableaux.
- `dist_min_type(type)` : retourne la distance minimale détectée pour un type d'objet.
- `dist_min_allie()` : retourne la distance minimale et l'index d'un allié proche.
- `ennemi_en_vue()` : indique si au moins un ennemi est détecté.
- `pression_ennemie_gauche_droite()` : calcule l'influence ennemie à gauche et à droite.
- `pression_alliee_gauche_droite()` : calcule l'influence des alliés pour la dispersion.
- `braitenberg_hateWall()` : évitement répulsif des murs pour une navigation sûre.
- `braitenberg_loveBot_ennemi()` : attraction vers les ennemis pour les repeindre.
- `braitenberg_hateBot()` : évitement des robots pour limiter les collisions.
- `evitement_allie_prioritaire(i)` : anti-collision allié basé sur les identifiants.
- `controleur_perceptrone_genetique()` : navigation fluide via perceptron optimisé par AG.